Architecture Overview

A high-level tour of the Document Intelligence Server's runtime components, for engineers ramping up on the system.

Request Flow

A document upload enters through the REST API, is parsed into Markdown, split into token-budgeted chunks, embedded (dense and sparse), and upserted into the vector store. Search requests skip parsing and chunking, going straight from query text to dense/sparse embeddings to a fused Qdrant lookup.

Components

- **FastAPI app** — serves the REST API and hosts the MCP server on the same process, mounted at `/mcp` over Streamable HTTP.
- **PostgreSQL** — stores document metadata and parsed text; chunk text itself lives only in Qdrant's payload, not duplicated in Postgres.
- **Qdrant** — stores dense and sparse vectors per chunk, with payload filtering by document id and tag, and performs the RRF fusion of both vector types at query time.
- **MCP server** — exposes the knowledge base as four agent-ready tools (`search` with optional tag and document filters, `list_documents`, `list_tags`, and `get_document` for reading a document's full text or specific chunks) gated by a static bearer token, separate from the unauthenticated REST API.

Deployment Topology

All four services (backend, frontend, Postgres, and Qdrant) run as separate containers under one docker-compose stack in local development and staging. Production splits the backend into multiple replicas behind a load balancer, while Postgres and Qdrant remain

single-instance — neither has been sharded yet, since document volume has not approached the point where that tradeoff is worth the added operational complexity.

Why No Alembic

Schema migrations run via `SQLModel.metadata.create_all` on startup rather than a dedicated migration tool. This is a deliberate, documented limitation: it works cleanly for additive schema changes (new tables, new nullable columns) but cannot express destructive changes (column drops, renames, type changes) without a manual one-off script. The tradeoff was accepted because the schema has been stable since the first release and the operational simplicity of skipping a migration tool outweighed the flexibility Alembic would add, at the current stage of the project.